

PROJECT DOCUMENTATION REPORT

1. Title and Project Overview

- **Project Title:** Your Librarian – The Midnight Archive (Library Management System)
- **Student Name:** A.Mohamed Rafi
- **Registration Number:** 922525148063
- **Department:** AIML-II
- **Project Repository:** <https://github.com/Rafi2618/Library-Management-System-CRUD>

Overview:

The Library Management System ("The Midnight Archive") is a complete full-stack CRUD web application developed in accordance with standard operating procedures. The project decouples the user interface from the backend server logic, communicating via standard RESTful API conventions. It allows administrators and library patrons to view book records, dynamically search and filter through genres, catalog new volumes, update details, and remove records with immediate persistence in a relational database.

2. Problem Statement

Traditional record-keeping in libraries is frequently inefficient, prone to manual entry errors, and lacks fast querying capabilities. Furthermore, monolithic architectures often blend business logic and user interface into a single layer, making scalability difficult. This project resolves these issues by presenting a responsive, separated client-server application offering instant data access, real-time validation, and automated data persistence.

3. Objectives

- Design and deploy a decoupled full-stack architecture using HTML, CSS, JavaScript, and Python Django REST Framework.
- Provide complete CRUD (Create, Read, Update, Delete) functionality with dynamic frontend feedback.

- Persist data reliably in an SQLite relational database utilizing Django ORM.
- Implement robust client-side validation to maintain data integrity.
- Provide an intuitive search and genre filtering mechanism.
- Validate all REST API endpoints through standard HTTP response testing.

4. Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript (ES6+)	Dynamic client-side user interface and async Fetch calls
Frontend Styling	Custom CSS (Luxury Dark-Gold Theme)	Theme styling, Google Fonts, and dynamic sparkle cursor
Backend Framework	Django 6.x / Django REST Framework	Server-side routing, serializers, and REST API controllers
Cross-Origin Handler	django-cors-headers	Handles Cross-Origin Resource Sharing (CORS)
Database	SQLite3	Persistent relational data storage via Django ORM
Testing & Tools	Web Browser / REST Client / VS Code	API inspection, runtime execution, and debugging

5. System Architecture

The application adheres strictly to the Decoupled Client-Server Web Architecture:

User Browser (Client) ↔ REST API Endpoints (Django) ↔ Django ORM
↔ SQLite Database

1. **Client Layer:** Renders the user interface, validates user inputs, and issues asynchronous HTTP requests (GET, POST, PUT, DELETE) with JSON payloads.

- 2. **REST API Layer:** Receives HTTP requests, validates data types via serializers, executes business logic, and responds with JSON and appropriate HTTP status codes.
- 3. **Database Layer:** Django ORM maps model classes into SQLite database tables and commits transactions.

6. Database Design & Schema

The primary entity managed by the system is the **Book** entity.

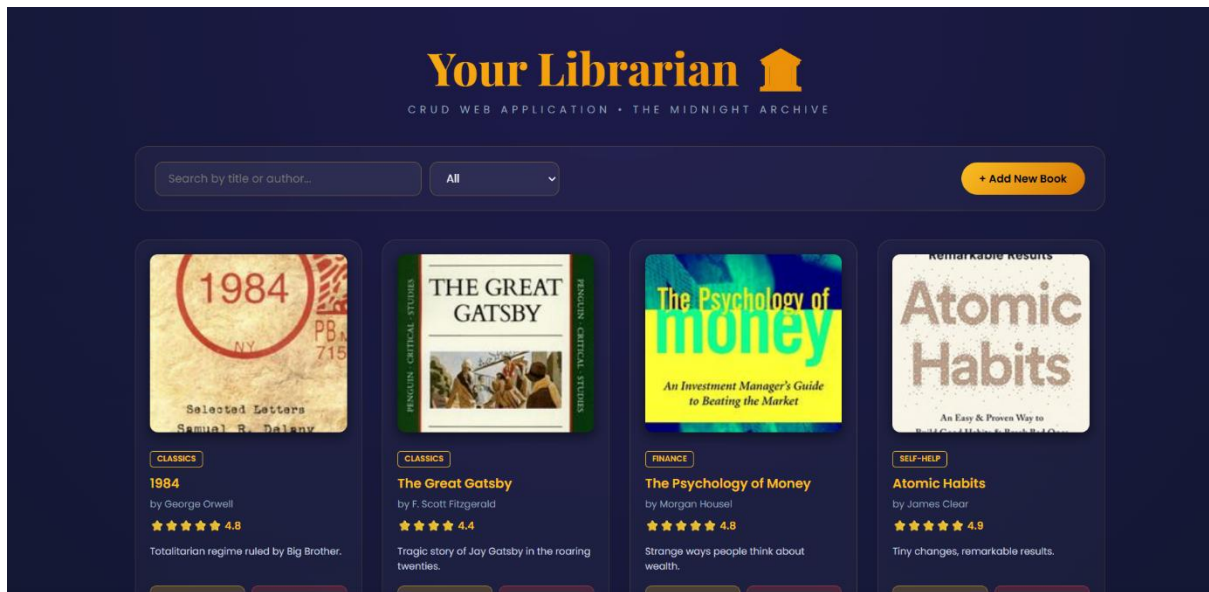
Entity Definition: books_book

Attribute	Data Type	Constraints	Description
id	Integer	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each volume
title	Varchar(200)	NOT NULL	Title of the volume
author	Varchar(150)	NOT NULL	Author of the volume
genre	Varchar(100)	NOT NULL	Classification or genre
rating	Float	NOT NULL, DEFAULT = 4.0	Rating score between 1.0 and 5.0
description	Text	Nullable / Blank Allowed	Detailed synopsis of the book

7. User Interface Screenshots

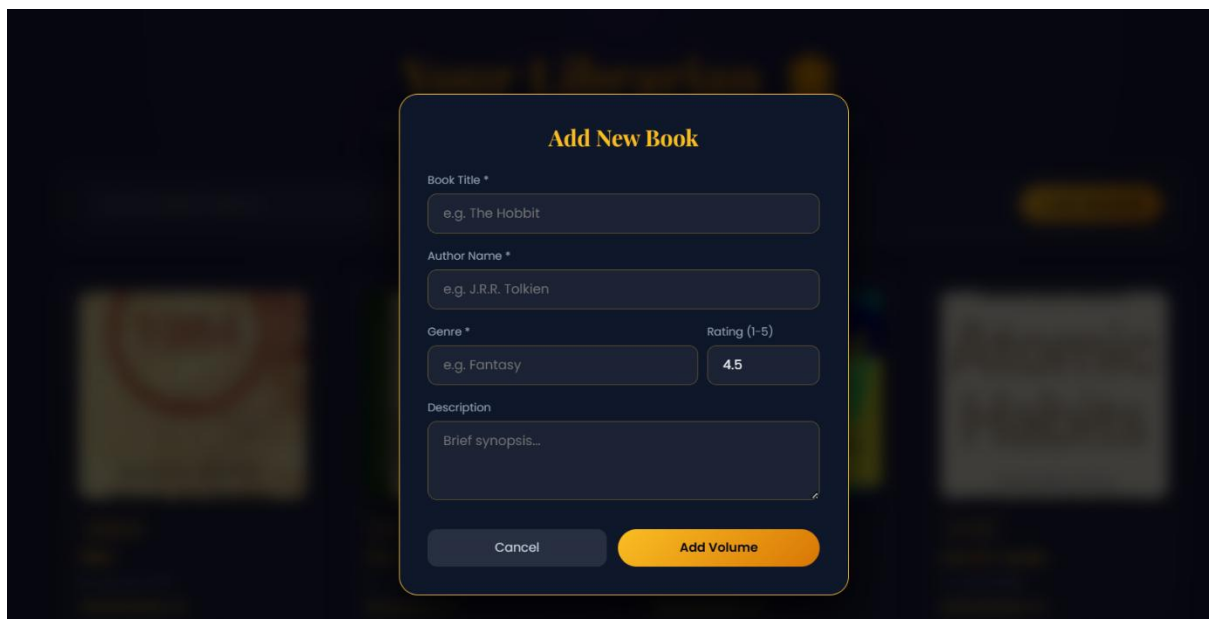
7.1 Home Screen (Catalog Overview & Live Cards)

(Displays all volumes with dynamic cover art, ratings, and genre tags)



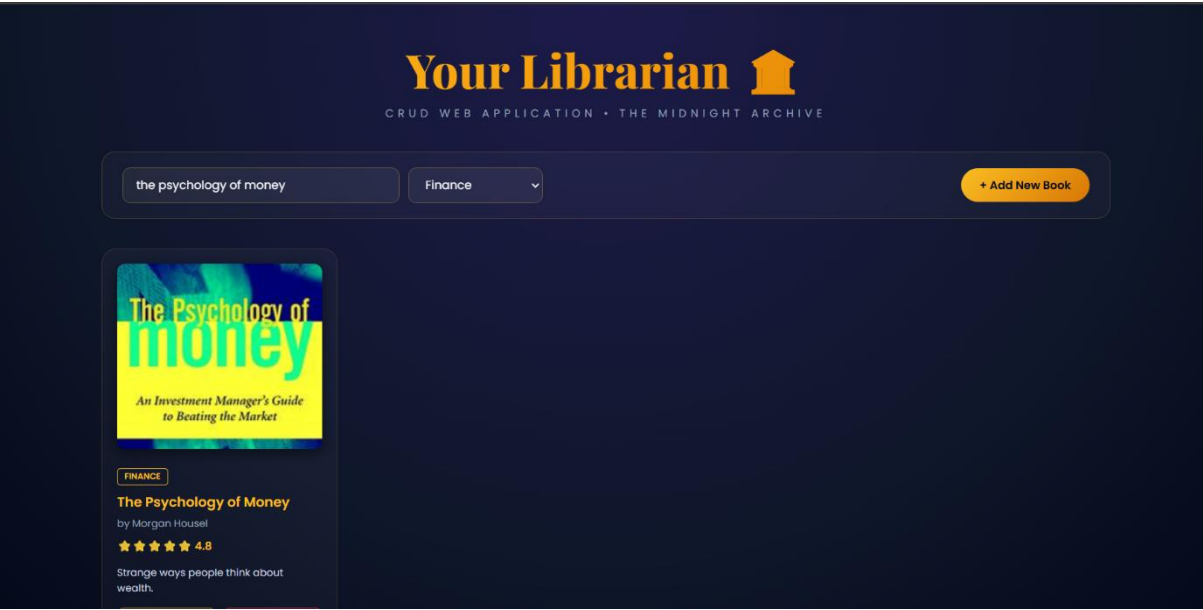
7.2 Create Record (Add New Book Modal)

(Form modal validating Title, Author, Genre, Rating, and Description)



7.3 Search & Category Filtering

(Live searching books by title or author and filtering by category)



8. REST API Endpoint Documentation

The backend exposes the following standard RESTful API endpoints:

Operation	HTTP Method	Endpoint URI	Request Body	Status Code	Expected Result
Read All	GET	/api/books/	None	200 OK	Retrieves an array of all book records
Read One	GET	/api/books/{id}/	None	200 OK	Retrieves details of a specific book by ID
Create	POST	/api/books/	JSON object	201 Created	Creates a new record in SQLite DB
Update	PUT	/api/books/{id}/	JSON object	200 OK	Updates all fields of an

					existing book
Delete	DELETE	/api/books/{id}/	None	204 No Content	Removes the record from the database

9. CRUD Implementation Details

- **Create (POST):** Users enter book details into the modal form. Upon submission, JavaScript packages the data into JSON and sends a POST request to /api/books/. The server saves the record, and the UI re-renders instantly.
 - **Read (GET):** When the application loads, loadBooks() triggers a GET request to /api/books/. Returned JSON records are parsed and mapped into dynamic luxury cards.
 - **Update (PUT):** Clicking the "Edit" button populates the modal with the selected book's current values. When submitted, a PUT request updates the record at /api/books/{id}/.
 - **Delete (DELETE):** Clicking "Delete" prompts user confirmation. Upon confirmation, a DELETE request is sent to /api/books/{id}/, removing the row from SQLite.
-

10. Testing Results & Verification

All REST endpoints and CRUD actions were tested:

Test Case	Tested Action	Input Data	Expected Output	Status
TC-01	Load Catalog	Page Load	Fetch all books via GET /api/books/	PASSED
TC-02	Add Book	Valid Book Data	Save to DB, return status 201 Created	PASSED

TC-03	Missing Fields	Empty Title/Author	Client-side validation stops form submit	PASSED
TC-04	Update Record	Edited Author/Rating	Database record updated, returns 200 OK	PASSED
TC-05	Delete Record	Click Delete & Confirm	Record removed, returns 204 No Content	PASSED

API Response Verification (Django REST Framework)

(Shows JSON response returned from the backend server)

[PASTE SCREENSHOT 4 HERE: Chrome-lab http://127.0.0.1:8000/api/books/ JSON page screenshot]

11. Installation and Execution Steps

Prerequisites

- Python 3.10+ installed
- Google Chrome or Microsoft Edge browser

Execution Steps

1. Open the project backend directory in the terminal:

```
codeBash
```

```
cd "min project/library_backend"
```

2. Install dependencies:

```
codeBash
```

```
pip install django djangorestframework django-cors-headers
```

3. Run database migrations:

```
codeBash
```

```
python manage.py migrate
```

4. Start the Django development server:

```
codeBash
```

```
python manage.py runserver
```

5. Open min project/index.html in any modern web browser to use the application.
-

12. Challenges and Solutions

1. CORS Blockage:

Problem: The browser blocked JavaScript fetch() calls due to origin restrictions between local files and the backend server (http://127.0.0.1:8000).

Solution: Integrated the django-cors-headers middleware and set CORS_ALLOW_ALL_ORIGINS = True in settings.py.

2. Real-Time UI Synchronization:

Problem: Changes made through Create, Update, or Delete were not immediately visible without manual browser reload.

Solution: Implemented async/await fetch handlers that invoke loadBooks() immediately after successful mutation calls.

13. Future Enhancements

- Implement User Authentication (JWT / Session-based) to separate Student and Librarian privileges.
 - Add automated book return due-date reminders and fine calculation logic.
 - Implement pagination for handling libraries with tens of thousands of volumes.
-

14. Git Repository and Reference Details

- **Project Repository:** <https://github.com/Rafi2618/Library-Management-System-CRUD>
- **Version Control Tool:** Git & GitHub Web
- **Primary Branch:** main
- **Commit History:** Working source code committed and pushed to remote repository.
- **Compliance:** Standard Operating Procedure (SOP) – Complete CRUD-Based Web Application Development Guidelines